Lecture 4 - Machine learning models

COMP3670/COMP6670 - Introduction to Machine Learning

Diego Marcondes

Semester 2 2026



Australian National University

Statistical estimation of $f^\star$

- Recall from Lecture 2 what is the target function in a binary classification problem:

$$f^\star(\mathbf{x}) \;=\; \mathbb{1}\{\eta(\mathbf{x}) > \tfrac{1}{2}\}, \qquad\qquad \eta(\mathbf{x}) \;=\; \mathbb{P}(Y = 1 \mid \mathbf{X} = \mathbf{x})$$

- This is a **classification rule**. Hand me η and I write down the optimal classifier at once: no hypothesis set, no search, no optimisation
- The one thing standing between us and $f^\star$ is that η is **unknown**, because P is unknown
- Which suggests an alternative to Empirical Risk Minimisation: **estimate η from the data, and substitute the estimate into the classification rule**

The plug-in rules

Build an estimate $\hat{\eta}$ of η from the sample $\mathcal{D}_N$, and return $\hat{h}(\mathbf{x}) = \mathbb{1}\{\hat{\eta}(\mathbf{x}) > \frac{1}{2}\}$. If $\hat{\eta}$ is close to η, then $\hat{h}$ behaves like $f^\star$.

Two modelling philosophies

	Plug-in rules (today, first half)	**Empirical Risk Minimisation** (Lectures 2–3)
what is assumed	a model for the **distribution** P	a hypothesis set $\mathcal{H}$ of **rules**
what the data is used for	to estimate the **distribution**	to compute $E_{\text{in}}(h)$ for each $h \in \mathcal{H}$
what is returned	the rule for $f^\star$, evaluated at the estimate	$\arg\min_{h \in \mathcal{H}} E_{\text{in}}(h)$
the loss function is	used only to *evaluate* the result afterwards	the thing being *minimised*
it works when	the distributional model is adequate	bias-variance trade-off controlled

- The left-hand column is the **statistician's** route: describe the phenomenon by estimating its distribution, then compute from that what is wanted, here the optimal classifier
- **A plug-in rule minimises no error over $\mathcal{H}$.** The training mistakes of $\hat{h}$ play no part in producing it; the data enters only through the estimate

Two ways to estimate η

- There are two families of methods, and they differ in **how much we are willing to assume**

Parametric (Chapter 4)

Assume the distribution has a *shape* fixed by a few unknown numbers. Estimate the numbers, plug them in

Nonparametric (Chapter 5)

Assume *no* shape. Estimate the distribution by *smoothing* the data, plug it in

- Assuming a shape means very few quantities to estimate, so **very little data is needed**: **provided the shape is right**
- Assuming nothing means the rule works for *any* P, at the cost of **much more data**
- In both cases the classifier is $\hat{h}(\mathbf{x}) = \mathbb{1}\{\hat{\eta}(\mathbf{x}) > \frac{1}{2}\}$: **the two families differ only in how $\hat{\eta}$ is built**

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Chapters 4 and 5.

Parametric classification

What "parametric" means

Definition (Parametric model)

A **parametric statistical model** is a family of distributions $\{p(\cdot \mid \boldsymbol{\theta}) : \boldsymbol{\theta} \in \Theta \subseteq \mathbb{R}^m\}$ indexed by **finitely many** numbers $\boldsymbol{\theta} = (\theta_1, \ldots, \theta_m)$. To assume the model is to assume that the unknown distribution is $p(\cdot \mid \boldsymbol{\theta}^\star)$, for some unknown $\boldsymbol{\theta}^\star \in \Theta$.

- Two families:
  - $Y \sim \text{Bernoulli}(\theta)$ if $\mathbb{P}(Y = 1) = \theta$ and $\mathbb{P}(Y = 0) = 1 - \theta$ with $\Theta = [0, 1]$
  - $Y \sim \text{Normal}(\mu, \sigma^2)$ with $\theta = (\mu, \sigma^2)$ and $\Theta = \mathbb{R} \times \mathbb{R}_+$.
- "Parametric" means **all our ignorance about the distribution is confined to finitely many numbers**
- A parametric plug-in rule has three steps, and **only the second looks at the data**: **(1)** choose the family; **(2)** estimate $\boldsymbol{\theta}^\star$ from $\mathcal{D}_N$; **(3)** substitute into the rule for $f^\star$

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Section 4.1.

Estimating the parameters of a statistical model

- A coin with unknown probability θ of heads is tossed 10 times, and we observe 7 **heads**. What is a sensible guess for θ?
- Try each candidate value in turn and ask: **if this were the truth, how probable would the thing we actually saw be?**

candidate θ	0.1	0.3	0.5	0.7	0.9
$\mathbb{P}(7 \text{ heads in } 10 \text{ tosses}) = 120\,\theta^7(1-\theta)^3$	0.00001	0.009	0.117	0.267	0.057

- $\theta = 0.1$ is all but ruled out: it would make our observation a one-in-a-hundred-thousand event. The value $\theta = 0.7$ makes it as probable as it can be
- **Idea**: Choosing the value that makes **the observed data as probable as possible** under the statistical model!

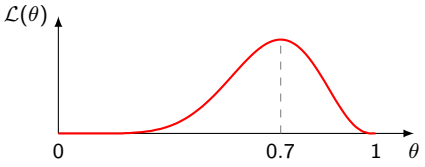
The likelihood function

Definition (Likelihood)

For an i.i.d. sample $Z_1, \ldots, Z_N$ from $p(\cdot \mid \boldsymbol{\theta}^\star)$, the **likelihood function** is the probability (or density) of the observed sample, read as a function of the parameter:

$$\mathcal{L}(\boldsymbol{\theta}) \;=\; p(Z_1 \mid \boldsymbol{\theta}) \times \cdots \times p(Z_N \mid \boldsymbol{\theta}) \;=\; \prod_{i=1}^{N} p(Z_i \mid \boldsymbol{\theta}).$$

- The *product* is due to **independence**
- **The data is fixed and θ varies.** It is the same formula as the joint distribution, read **as a function of θ**
- $\mathcal{L}$ is *not* a distribution over $\boldsymbol{\theta}$: it does not integrate to 1, and $\boldsymbol{\theta}$ is not random



For the coin, $\mathcal{L}(\theta) = 120\,\theta^7(1-\theta)^3$

Maximum likelihood estimation

Maximum likelihood estimator

$$\hat{\boldsymbol{\theta}} = \arg\max_{\boldsymbol{\theta} \in \Theta} \mathcal{L}(\boldsymbol{\theta}), \qquad \mathcal{L}(\boldsymbol{\theta}) = \prod_{i=1}^{N} p(Z_i \mid \boldsymbol{\theta})$$

- In simple families $\hat{\boldsymbol{\theta}}$ has a closed form. In general it does not, and it is found **numerically**
- **Bernoulli**(θ), with $k = Z_1 + \cdots + Z_N$ successes out of N trials $\hat{\theta} = \frac{k}{N}$, the **observed proportion** of successes, which is $7/10$ in the coin example
- **Normal**(μ, σ^2), the **sample mean** and the **sample variance**:

$$\hat{\mu} = \frac{1}{N} \sum_{i=1}^{N} Z_i, \qquad \hat{\sigma}^2 = \frac{1}{N} \sum_{i=1}^{N} (Z_i - \hat{\mu})^2$$

- **In Weeks 11-12 we will learn how to compute maximum likelihood estimators**

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Chapter 4.

The guarantees of maximum likelihood

- **If the family is correct**: under mild conditions $\hat{\theta}$ converges to $\theta^{\star}$ as N grows, and no other estimator is asymptotically more precise. It extracts everything the sample contains
- **If the family is wrong**: $\hat{\theta}$ converges to the member of the family *closest* to the truth, and the procedure **never notices**. Nothing in $\mathcal{L}$ measures whether the family was a good idea in the first place
- That is the **approximation error** of Lecture 3 in a different context: a small family is cheap to estimate, and may be irreparably wrong

Maximum likelihood

Among the distributions we were **willing to consider**, return the one under which what actually happened was most probable.

Back to classification: a parametric model for η

- We do not need all of P: the rule for $f^\star$ asks only for $\eta(\mathbf{x}) = \mathbb{P}(Y = 1 \mid \mathbf{X} = \mathbf{x})$. So let us put a parametric model **directly on** η
- The constraints: $\eta(\mathbf{x})$ must lie in $[0, 1]$ and, of course, must be a function of $\mathbf{x}$. The simplest function of $\mathbf{x}$ is the linear function $\sum_{j=1}^{d} w_j x_j + w_0$, but that is **not** in $[0, 1]$
- **The fix**: transform the probability first. The **logit** maps a probability p in $(0, 1)$ onto the whole real line,

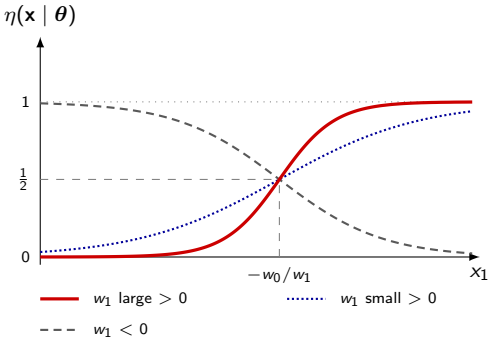
$$\text{logit}(p) \;=\; \log \frac{p}{1-p}, \qquad 0 < p < 1$$

  so "p small" becomes "very negative", "$p = \frac{1}{2}$" becomes "0", "p near 1" becomes "very positive". It is the log of the **odds**
- **The model**: the logit is a weighted score of the features with $\boldsymbol{\theta} = (w_0, w_1, \ldots, w_d)$,

$$\log \frac{\eta(\mathbf{x} \mid \boldsymbol{\theta})}{1 - \eta(\mathbf{x} \mid \boldsymbol{\theta})} \;=\; \sum_{j=1}^{d} w_j x_j + w_0 \qquad \Longleftrightarrow \qquad \eta(\mathbf{x} \mid \boldsymbol{\theta}) \;=\; \frac{1}{1 + e^{-(\sum_{j=1}^{d} w_j x_j + w_0)}}$$

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Section 4.3, equations (4.28)–(4.30).

The logistic curve



One feature ($d = 1$), so $\eta(\mathbf{x} \mid \boldsymbol{\theta}) = \left(1 + e^{-(w_1 x_1 + w_0)}\right)^{-1}$, and all three curves share a crossing point.

- An S-shaped curve, increasing from 0 to 1, crossing $\frac{1}{2}$ exactly where $\sum_{j=1}^{d} w_j x_j + w_0 = 0$
- The sign of the weight w_j says whether the probability of class 1 **increases or decreases** with variable j; its magnitude says how sharply
- The model says **nothing** about how the features are distributed inside each class. It describes only *how the probability of class* 1 *varies across feature space*
- That is a much weaker assumption than modelling the whole of P, but it is all we need in the plug-in rule

Fitting the model: one coin per "individual"

- Given $\mathbf{X}_i$, the label Y_i is a **single coin toss** with probability of heads $\eta(\mathbf{X}_i \mid \boldsymbol{\theta})$: **every individual has their own probability** of $Y = 1$, and the model ties these probabilities together through the same $d + 1$ numbers $\boldsymbol{\theta}$
- So the likelihood of the observed labels is

$$\mathcal{L}(\boldsymbol{\theta}) \;=\; \prod_{i=1}^{N} \eta(\mathbf{X}_i \mid \boldsymbol{\theta})^{Y_i} \left(1 - \eta(\mathbf{X}_i \mid \boldsymbol{\theta})\right)^{1-Y_i}$$

  each factor being $\eta(\mathbf{X}_i)$ when $Y_i = 1$, and $1 - \eta(\mathbf{X}_i)$ when $Y_i = 0$
- Taking logarithms gives the **log-likelihood** (**recall the properties of logarithms!**)

$$\mathrm{LL}(\boldsymbol{\theta}) \;=\; \log \mathcal{L}(\boldsymbol{\theta}) \;=\; \sum_{i=1}^{N} \Big[\, Y_i \log \eta(\mathbf{X}_i \mid \boldsymbol{\theta}) \;+\; (1 - Y_i) \log \left(1 - \eta(\mathbf{X}_i \mid \boldsymbol{\theta})\right) \Big]$$

- Maximising LL is the same as minimising $-\mathrm{LL}/N$: and $-\mathrm{LL}/N$ is an **average over the sample** of a "loss" computed at each observation

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Section 4.3, equation (4.31).

A new loss: the Cross-entropy loss

- That quantity computed at each observation has a name, and it is a **loss function**

Cross-entropy loss

For a hypothesis that outputs a **probability** $\eta(\mathbf{x}) \in (0, 1)$ rather than a label,

$$\ell\big((\mathbf{x}, y), \eta\big) \;=\; -\,y \log \eta(\mathbf{x}) \;-\; (1-y) \log\big(1 - \eta(\mathbf{x})\big)$$

- Read it: if $y = 1$ we pay $-\log \eta(\mathbf{x})$, which is 0 when we said "certainly 1" and $+\infty$ when we said "certainly 0". Symmetrically for $y = 0$
- So **maximum likelihood for the logistic model is Empirical Risk Minimisation with the cross-entropy loss**:

$$\hat{\boldsymbol{\theta}} \;=\; \arg\max_{\boldsymbol{\theta}} \; \mathrm{LL}(\boldsymbol{\theta}) \;=\; \arg\min_{\boldsymbol{\theta}} \; \frac{1}{N} \sum_{i=1}^{N} \ell\big((\mathbf{X}_i, Y_i), \eta(\cdot \mid \boldsymbol{\theta})\big)$$

- Unlike the coin tossing example, there is **no closed form** to the parameter value and we need to compute it numerically (Weeks 11-12)

Why a new loss at all?

- The binary loss asks only *was the predicted label right?* It cannot handle hypothesis that return the probability of the label one, instead of a label, e.g., that said "51% sick" or "99% sick", when the patient was in fact sick
- The cross-entropy loss scores the **whole probability statement**. A confident mistake is greatly punished; a hesitant one, mildly. That is what makes it usable for *fitting* a probability model
- A classifier is obtained from $\hat{\theta}$ by the plugin-rule

$$\hat{h}(\mathbf{x}) = \mathbb{1}\left\{ \frac{1}{1 + e^{-(\sum_{j=1}^{d} \hat{w}_j x_j + \hat{w}_0)}} > 1/2 \right\} = \mathbb{1}\left\{ \sum_{j=1}^{d} \hat{w}_j x_j + \hat{w}_0 > 0 \right\}$$

Surrogate losses

We *care* about the binary loss. We *minimise* something else, chosen so that making it small tends to make the binary loss small. Such a stand-in is called a **surrogate loss**.

What the logistic hypothesis means

- The fitted model returns a probability, and the plug-in rule compares it with $\frac{1}{2}$:

$$\hat{h}(\mathbf{x}) = \mathbb{1}\left\{\frac{1}{1+e^{-(\sum_{j=1}^{d}\hat{w}_j x_j + \hat{w}_0)}} > 1/2\right\} = \mathbb{1}\left\{\sum_{j=1}^{d}\hat{w}_j x_j + \hat{w}_0 > 0\right\}$$

- So, perhaps surprisingly, **the logistic classifier is a linear classifier**: the S-shaped curve disappears entirely once the threshold is applied
- In words, $\hat{h}$ is **the linear classifier whose associated probability model makes the observed labels most likely**
- The probabilities themselves are often the real output: "$\hat{\eta}(\mathbf{x}) = 0.83$" is far more useful to a clinician than "class 1"
- **Note**: it gives another algorithm to train a perceptron! But now the goal is to minimise another loss (cross-entropy), and not the classification error.

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Section 4.3, equation (4.33).

There is actually a hypothesis set!

- As $\boldsymbol{\theta}$ ranges over all its possible values, it sweeps out a set of classifiers:

$$\mathcal{H} \;=\; \left\{ \mathbf{x} \mapsto \mathbb{1}\{w_1 x_1 + \cdots + w_d x_d + w_0 > 0\} \;:\; w_0, w_1, \ldots, w_d \in \mathbb{R} \right\}$$

  the perceptron!

- So the choice of **parametric family led** to a hypothesis set: we never chose $\mathcal{H}$, it is a consequence of the statistical model
- The point of view of statistics, that is modelling $\eta(\mathbf{x})$, leads to another loss function for learning in this hypothesis set, the cross-entropy loss
- This is a common feature of maximum likelihood estimation under a model $\eta(\mathbf{x}|\boldsymbol{\theta})$

Back to the credit approval problem

- The bank of Lecture 1 records $d = 3$ features. To keep the numbers readable we measure money in **units of \$10,000**:

$$x_1 = \text{annual income}, \qquad x_2 = \text{years employed}, \qquad x_3 = \text{outstanding debt}$$

- We have $Y = 1$ means the applicant **defaults** and $Y = 0$ means they **repay**. So $\eta(\mathbf{x})$ is the **probability of default**
- The bank fits the logistic model to the data of past applicants by maximum likelihood, i.e. by minimising the cross-entropy loss. Suppose it obtains

The fitted model

$$\hat{w}_0 = 1.20, \qquad \hat{w}_1 = -0.60, \qquad \hat{w}_2 = -0.40, \qquad \hat{w}_3 = +1.00$$

$$\hat{\eta}(\mathbf{x}) \;=\; \frac{1}{1 + e^{-(1.20 - 0.60x_1 - 0.40x_2 + 1.00x_3)}}$$

- **These four numbers are the entire model**: the data of past applicants has been compressed into them

What the score is: the log-odds

- Write $g(\mathbf{x}) = \hat{w}_0 + \hat{w}_1 x_1 + \hat{w}_2 x_2 + \hat{w}_3 x_3$ for the score, so that $\hat{\eta}(\mathbf{x}) = 1/(1 + e^{-g(\mathbf{x})})$. Then

$$1 - \hat{\eta}(\mathbf{x}) \;=\; 1 - \frac{1}{1 + e^{-g(\mathbf{x})}} \;=\; \frac{\left(1 + e^{-g(\mathbf{x})}\right) - 1}{1 + e^{-g(\mathbf{x})}} \;=\; \frac{e^{-g(\mathbf{x})}}{1 + e^{-g(\mathbf{x})}}$$

- Dividing one by the other, the denominators cancel:

$$\frac{\hat{\eta}(\mathbf{x})}{1 - \hat{\eta}(\mathbf{x})} \;=\; \frac{1}{1 + e^{-g(\mathbf{x})}} \times \frac{1 + e^{-g(\mathbf{x})}}{e^{-g(\mathbf{x})}} \;=\; \frac{1}{e^{-g(\mathbf{x})}} \;=\; e^{g(\mathbf{x})}$$

- That ratio is called the **odds** of default. Taking logarithms of both sides,

$$\log \frac{\hat{\eta}(\mathbf{x})}{1 - \hat{\eta}(\mathbf{x})} \;=\; g(\mathbf{x}) \;=\; \hat{w}_0 + \hat{w}_1 x_1 + \hat{w}_2 x_2 + \hat{w}_3 x_3$$

- So **the score is exactly the log-odds of default**. This is why the model is linear in a useful sense, even though $\hat{\eta}$ itself is an S-shaped curve

What one unit of a feature does

- Take an applicant $\mathbf{x}$ and add 1 to feature j, leaving the others fixed. The score changes by exactly $\hat{w}_j$, since $\hat{w}_j(x_j + 1) = \hat{w}_j x_j + \hat{w}_j$
- By the previous slide the odds are e^{score}, so the **new** odds are

$$e^{g(\mathbf{x}) + \hat{w}_j} \;=\; e^{g(\mathbf{x})} \times e^{\hat{w}_j} \;=\; (\text{old odds}) \times e^{\hat{w}_j}$$

- **Adding one unit of feature j multiplies the odds of default by $e^{\hat{w}_j}$**, whatever the applicant was to begin with. Working the three numbers out:
  - $\hat{w}_1 = -0.60$, so $e^{-0.60} \approx 0.55$: each extra \$10,000 of **income** roughly **halves** the odds of default
  - $\hat{w}_2 = -0.40$, so $e^{-0.40} \approx 0.67$: each extra **year employed** cuts the odds by about a third
  - $\hat{w}_3 = +1.00$, so $e^{1.00} \approx 2.72$: each extra \$10,000 of **debt** nearly **triples** the odds
- The two negative coefficients say income and steady employment **push the risk of default down**; the positive one says debt **pushes it up**. A coefficient of 0 would mean the feature is irrelevant once the others are known

Three applicants

	Income x_1	Years x_2	Debt x_3	Score $\hat{w}_0 + \hat{w}_1 x_1 + \hat{w}_2 x_2 + \hat{w}_3 x_3$	$\hat{\eta}(\mathbf{x})$ **P(default)**	Decision $\hat{h}(\mathbf{x})$
Ana	\$52,000	5	\$3,200	$1.20 - 0.60(5.2) - 0.40(5) + 1.00(0.32) = -3.60$	0.027	**approve**
Ben	\$28,000	1	\$15,400	$1.20 - 0.60(2.8) - 0.40(1) + 1.00(1.54) = 0.66$	0.659	**deny**
Cleo	\$34,000	2	\$15,000	$1.20 - 0.60(3.4) - 0.40(2) + 1.00(1.50) = -0.14$	0.465	**approve**

Money enters in units of \$10,000, so Ana's \$52,000 is $x_1 = 5.2$ and her \$3,200 of debt is $x_3 = 0.32$. Then $\hat{\eta} = 1/(1 + e^{-\text{score}})$, and the plug-in rule denies credit exactly when $\hat{\eta} > \frac{1}{2}$, that is when the score is positive.

- **Ana** is the applicant who repaid in Lecture 1: good income, five years employed, almost no debt. Every term pushes the score down, and 2.7% is a comfortable approval
- **Ben** is the applicant who defaulted: his debt alone contributes $+1.54$, and it is not offset. The model puts him at 66% and denies

The third applicant is the interesting one

- **Cleo** earns \$6,000 more than Ben and has been employed a year longer, but carries almost the same debt. Her score is -0.14: **barely** on the approve side
- The plug-in rule reports the same thing for Ana and for Cleo (*approve*) but $\hat{\eta} = 0.027$ and $\hat{\eta} = 0.465$ are **not the same statement at all**. Cleo is close to a coin toss
- **This is the point of modelling η rather than $f^\star$.** The binary loss cannot tell these two applicants apart; the cross-entropy loss was built precisely so that it can
- A bank would not use the threshold $\frac{1}{2}$ anyway: a default costs it far more than a refused customer, so it would deny above, say, $\hat{\eta} > 0.15$, and then Cleo is denied and Ana is not

Why the probability is the useful output

The threshold encodes **what a mistake costs**, and that is a decision about the world, not about the data. Estimating η separates the two: the model supplies the probability, the bank supplies the threshold.

Nonparametric classification

Dropping the shape assumption

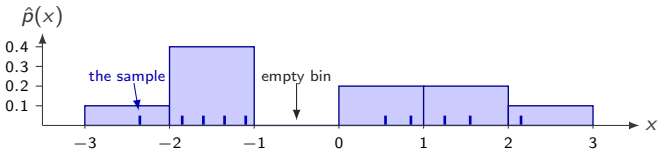
- Parametric rules are efficient to estimate because they assume a lot: a family of distributions for $\eta(\mathbf{x})$
- **Nonparametric** rules assume nothing about the shape of the distributions
- Instead of a formula with a few parameters, they build the estimate of η by **smoothing the data**: look near $\mathbf{x}$, and see which class is in the majority there
- What is gained: the ability to produce **arbitrarily complicated** decision boundaries, and to work for *any* distribution
- What is paid: much **more data**, and a serious risk of overfitting
- And a new ingredient appears in every one of these rules: a **smoothing parameter** controlling how large "near $\mathbf{x}$" is

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Section 5.1.

Estimating a distribution without a shape: the histogram

- Approximating densities with histograms: with N observations and bins of width l,

$$\hat{p}(x) \;=\; \frac{\#\{i \,:\, X_i \text{ lies in the bin containing } x\}}{N \times l}$$



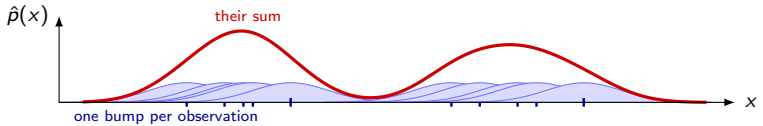
Here $N = 10$ and $l = 1$, so each bar has height (number of sample points in that bin)/10.

- It is a **step function**: flat inside a bin, jumping at the edges. Two observations 0.01 apart may fall on opposite sides of an edge and never see one another
- It depends on **where** the edges were drawn, not only on how wide the bins are, and bins can come out **empty** in the middle of the data
- The estimate is jagged; the phenomenon presumably is not. **We want something smoother**

From bins to bumps: the kernel idea

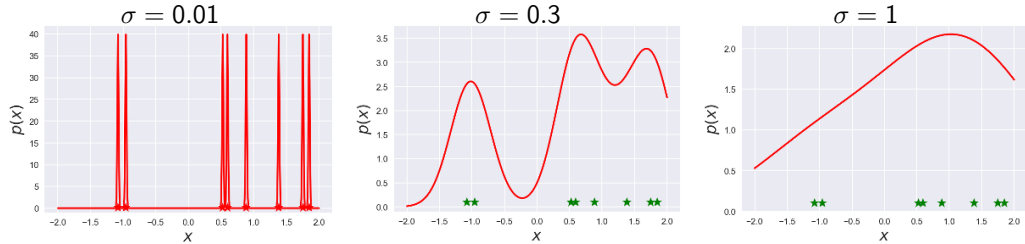
- In the histogram each observation contributes a **rectangle over its own bin**: full weight everywhere inside it, none outside. That abruptness is the whole problem
- **The idea**: let each observation contribute a small **bump centred on itself** instead, and add the bumps up

$$\hat{p}(\mathbf{x}) \;=\; \frac{1}{N\sigma^d} \sum_{i=1}^{N} \kappa\left(\frac{\mathbf{x} - \mathbf{X}_i}{\sigma}\right), \qquad \text{for instance} \quad \kappa(u) \;=\; \frac{1}{\sqrt{2\pi}}\, e^{-u^2/2}$$



- Every observation now influences the estimate **everywhere**, with a strength that fades with distance: $\hat{p}(\mathbf{x})$ is a **smoothing of the observations close to x**
- The **kernel** κ is the shape of the bump; σ is its **width**, and it replaces the bin width. σ **is the smoothing parameter**: it says how far "close to **x**" reaches

Too little, about right, too much



- A density estimated from the same data by smoothing, with three amounts of smoothing
- **Left**: a spike on every observation. The estimate reproduces the sample, not the phenomenon **(overfitting)**
- **Right**: one broad hump. All the structure has been smoothed away **(underfitting)**
- **Middle**: about right. And "about right" is a **model selection** problem, in the sense of Lecture 3

Source: Braga-Neto (2024), Figure 5.1 (Example 5.1).

From the density of $\mathbf{X}$ to the conditional probability η

- Smoothing gave us $p(\mathbf{x})$, the density of the **features alone**. The plug-in rule does not ask for that: it asks for $\eta(\mathbf{x}) = \mathbb{P}(Y = 1 \mid \mathbf{X} = \mathbf{x})$
- **The same idea, applied twice**: smooth the class-1 and the class-0 observations separately,

$$\hat{p}_1(\mathbf{x}) = \frac{1}{N\sigma^d} \sum_{i=1}^{N} \kappa\left(\frac{\mathbf{x} - \mathbf{X}_i}{\sigma}\right) \mathbb{1}\{Y_i = 1\}, \qquad \hat{p}_0(\mathbf{x}) = \frac{1}{N\sigma^d} \sum_{i=1}^{N} \kappa\left(\frac{\mathbf{x} - \mathbf{X}_i}{\sigma}\right) \mathbb{1}\{Y_i = 0\}$$

- The first estimates the "density of $\mathbf{x}$ *and* class 1", which is $p(\mathbf{x})\eta(\mathbf{x})$; the second, $p(\mathbf{x})(1 - \eta(\mathbf{x}))$; their sum, $p(\mathbf{x})$. By the definition of **conditional probability**:

$$\hat{\eta}(\mathbf{x}) \;=\; \frac{\hat{p}_1(\mathbf{x})}{\hat{p}_1(\mathbf{x}) + \hat{p}_0(\mathbf{x})} \;=\; \sum_{i=1}^{N} W_i(\mathbf{x})\, \mathbb{1}\{Y_i = 1\}, \qquad W_i(\mathbf{x}) = \frac{\kappa\big((\mathbf{x} - \mathbf{X}_i)/\sigma\big)}{\sum_{j=1}^{N} \kappa\big((\mathbf{x} - \mathbf{X}_j)/\sigma\big)}$$

- In words: a **density** estimate asks *how many* observations are near $\mathbf{x}$; a **conditional** estimate asks *what fraction of them carry the label* 1. **We smooth the labels, not the points**

Nonparametric classification rules

- Every rule in this section estimates $\eta(\mathbf{x})$ as a **weighted vote** of the training labels

The general nonparametric plug-in rule

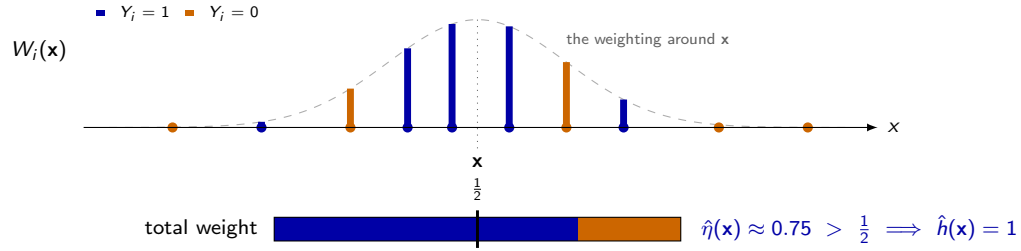
$$\hat{\eta}(\mathbf{x}) \;=\; \sum_{i=1}^{N} W_i(\mathbf{x})\, \mathbb{1}\{Y_i = 1\}, \qquad\qquad W_i(\mathbf{x}) \geq 0, \quad \sum_{i=1}^{N} W_i(\mathbf{x}) = 1$$

$$\hat{h}(\mathbf{x}) \;=\; \mathbb{1}\{\hat{\eta}(\mathbf{x}) > \tfrac{1}{2}\} \;=\; \mathbb{1}\{\sum_{i=1}^{N} W_i(\mathbf{x}) \mathbb{1}\{Y_i = 1\} \;>\; \sum_{i=1}^{N} W_i(\mathbf{x}) \mathbb{1}\{Y_i = 0\}\}$$

- $W_i(\mathbf{x})$ is the **influence** of observation i on the decision at the point $\mathbf{x}$. The weights depend on $\mathbf{x}$, and also on where *all* the other observations are
- **The rules of this section differ only in how the weights are defined**

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Section 5.1, equations (5.1)–(5.3).

The weighted vote, in a picture



- Each training point gets a weight $W_i(\mathbf{x})$, drawn as a bar: **tall when close to x**, short when far. The bars add up to 1
- Stack them by label: the blue part is $\hat{\eta}(\mathbf{x})$, the orange part $1 - \hat{\eta}(\mathbf{x})$, and $\hat{h}(\mathbf{x})$ is decided by **which side of the midpoint the split falls on**
- The *nearest* points do not decide on their own: **a distant crowd can outvote a close pair**, depending on how fast the weights decay
- Move **x** and every weight changes: **the classifier is rebuilt at each point where it is evaluated**

The assumption that replaced the shape assumption

- In every case the weight $W_i(\mathbf{x})$ is **large when $\mathbf{X}_i$ is close to $\mathbf{x}$** and small when it is far
- That encodes an assumption about the phenomenon:

Spatial coherence

Points that are **close together in feature space are more likely to belong to the same class** than points that are far apart.

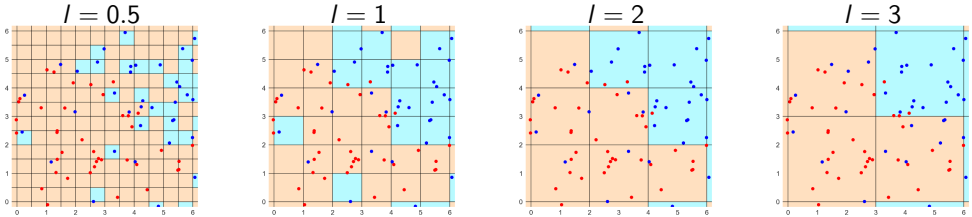
- So nonparametric does **not** mean assumption-free. It means we traded a strong assumption about *shape* for a weak assumption about *locality*
- And the assumption can fail: if a variable is measured in the wrong units, or is irrelevant, "close" means the wrong thing. **These methods are sensitive to how the features are scaled**
- It also degrades badly when d is large, because in high dimension nothing is close to anything

What is $\mathcal{H}$ for a nonparametric rule?

- There is no list of parameters, so we cannot write $\mathcal{H} = \{h_{\boldsymbol{\theta}}\}$
- $\mathcal{H}$ is the collection of *all* classifiers the rule could output, over all possible datasets of size N and it **grows with** N
- So the model is **not fixed in advance**: more data buys not only a better estimate but a *richer* hypothesis set. That is precisely what lets these rules approach $f^\star$ for *any* P
- The Lecture 3 picture still applies, with a twist: the smoothing parameter is what **keeps the complexity in check** at a given N. Less smoothing $\Rightarrow$ effectively larger $\mathcal{H}$ $\Rightarrow$ more variance

Nonparametric rules are also **plug-in** *rules: no error over $\mathcal{H}$ is minimised. They estimate $\hat{\eta}$, then imitate $f^\star$.*

The cubic histogram rule



- **l too small**: most cells contain one point or none, so the estimate of η is meaningless there. Empty cells are decided by an arbitrary tie-breaking convention
- **l too large**: too few cells to describe the boundary at all
- **The deeper problem**: the grid is axis-aligned, and the true boundary is not. **No** value of l fixes that

Source: Braga-Neto (2024), Figure 5.2 (Example 5.2).

General histogram classification

- **The idea**: chop the feature space into non-overlapping cells that cover everything, a **partition**, and let each cell decide by majority vote
- Write $A(\mathbf{x})$ for the cell containing $\mathbf{x}$, and $N(\mathbf{x})$ for the number of observations in it. Then

$$W_i(\mathbf{x}) = \begin{cases} \dfrac{1}{N(\mathbf{x})}, & \mathbf{X}_i \in A(\mathbf{x}) \\ 0, & \text{otherwise} \end{cases}$$

- Only the points in *the same cell* as $\mathbf{x}$ vote, and they vote **equally**
- So $\hat{h}$ assigns to every cell the **majority label of the training points inside it**, and is constant on each cell
- In the **cubic histogram rule** the partition is a regular grid of cubes of side l. **l is the smoothing parameter**

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Section 5.2, equations (5.4)–(5.6).

k-nearest-neighbour classification

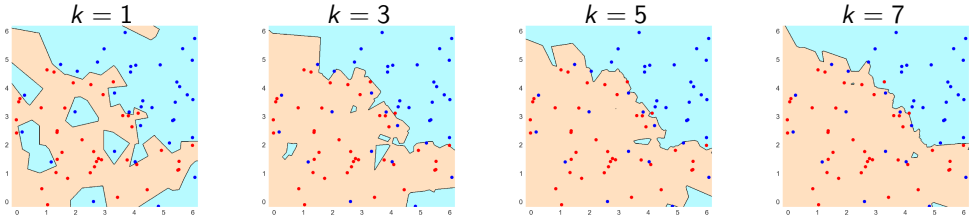
- **The idea**: instead of fixing the *region* and counting whatever points fall in it, fix the *number of points* and let the region adapt
- $\hat{h}(\mathbf{x})$ is the **majority label among the k training points nearest to $\mathbf{x}$**, that is

$$W_i(\mathbf{x}) = \begin{cases} \frac{1}{k}, & \mathbf{X}_i \text{ is among the } k \text{ nearest neighbours of } \mathbf{x} \\ 0, & \text{otherwise} \end{cases}$$

- Two immediate advantages over the histogram: the neighbourhood **adapts to the shape and orientation** of the data, and **ties disappear** if k is odd
- $k = 1$ gives the **nearest-neighbour rule**: copy the label of the closest training point
- **k is the smoothing parameter**, and it works *backwards*: small k means little smoothing

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Section 5.3, equation (5.7).

k-nearest-neighbour classification



- Same data as before. $k = 1$ classifies every training point correctly by construction ($E_{\text{in}} = 0$) and the islands around isolated points are pure overfitting
- $k = 3$ **and** $k = 5$ are far better, and $k = 5$ is already close to the optimal boundary
- $k = 7$ improves little on $k = 5$
- Unlike the histogram, the boundary **is free to follow the diagonal**

Source: Braga-Neto (2024), Figure 5.3 (Example 5.3).

What the kNN hypothesis means

- $\hat{h}$ is a **local majority vote**. There is no formula, no parameter, no summary of the data: **the training set is the model**
- "Training" costs nothing: one just stores the data. *Predicting* is what costs, since every new point requires searching for its neighbours
- E_{in} is a particularly poor guide here: at $k = 1$ it is **exactly zero** for any dataset whatsoever
- In practice $k = 3$ or $k = 5$ are good default choices, and k should be selected by cross-validation
- Nearest-neighbour rules are not just a heuristic: for large N the error of the 1NN rule is at most **twice** the error of the best possible classifier, whatever the distribution. That is the Cover–Hart Theorem

See: Braga-Neto (2024), Section 5.5 (Cover–Hart Theorem)

Kernel classification

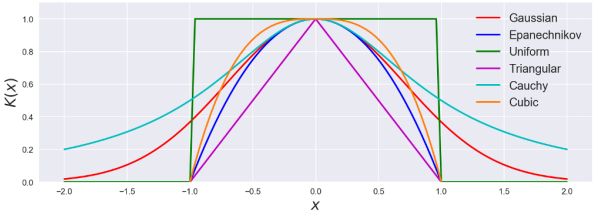
- Both previous rules give **equal** weight to the voters and **zero** to everyone else. That is abrupt. Kernel rules let the weight decay **smoothly** with distance: this is the rule we already met when smoothing a density
- A **kernel** is a non-negative function $\kappa : \mathbb{R}^d \to \mathbb{R}$, usually decreasing in $\|\mathbf{x}\|$. If a kernel depends on its input only through its norm, it is called a **radial basis function**
- A kernel induces the weights (with the *same* σ that set the width of the bumps above)

$$W_i(\mathbf{x}) \;=\; \frac{\kappa\left(\dfrac{\mathbf{x}-\mathbf{X}_i}{\sigma}\right)}{\sum_{j=1}^{N}\kappa\left(\dfrac{\mathbf{x}-\mathbf{X}_j}{\sigma}\right)}$$

- Every observation votes, with a strength that fades with distance. σ **is the bandwidth**, the smoothing parameter: it sets the distance scale over which the fading happens
- The denominator is there only to make the weights sum to 1

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Section 5.4, equations (5.8)–(5.9).

Some kernels

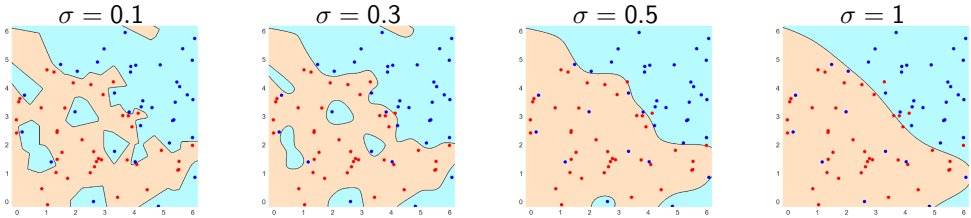


Source: Braga-Neto (2024), Figure 5.4.

Gaussian: $\kappa(\mathbf{x}) = e^{-\|\mathbf{x}\|^2}$
Cauchy: $\kappa(\mathbf{x}) = \left(1 + \|\mathbf{x}\|^{d+1}\right)^{-1}$
Uniform: $\kappa(\mathbf{x}) = \mathbb{1}\{\|\mathbf{x}\| \leq 1\}$
Triangular: $\kappa(\mathbf{x}) = (1 - \|\mathbf{x}\|)\mathbb{1}\{\|\mathbf{x}\| \leq 1\}$
Epanechnikov: $\kappa(\mathbf{x}) = (1 - \|\mathbf{x}\|^2)\mathbb{1}\{\|\mathbf{x}\| \leq 1\}$

- All but the Gaussian and the Cauchy are **zero outside a ball**: distant points are ignored entirely
- The uniform kernel gives every point within distance σ an equal vote: a moving histogram
- **The bandwidth σ matters far more than the kernel**

Kernel classification in action



- Gaussian kernel, same data again. Compare panel by panel with the kNN pictures
- $\sigma = 0.1$ looks almost exactly like 1NN. At such a small bandwidth the nearest point outvotes everyone else, and overfits in the same way
- As σ grows the boundaries become **visibly smoother** than the kNN ones. This is the effect of the gradual weights
- $\sigma = 1$ is close to the optimal line. Larger still and far-away points will start to dominate: **underfitting**

Source: Braga-Neto (2024), Figure 5.5 (Example 5.4).

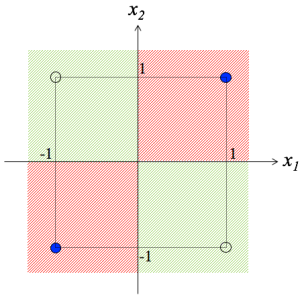
The three nonparametric rules

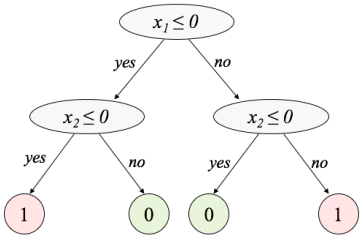
	Histogram	**kNN**	**Kernel**
who votes	points in the same cell	the k closest points	everyone
with what weight	equal	equal	decaying with distance
smoothing parameter	cell side l	number of neighbours k	bandwidth σ
less smoothing means	l small	k small	σ small
boundary	axis-aligned steps	jagged, data-adapted	smooth, data-adapted
main weakness	fixed grid orientation	abrupt weights	cost, and choice of σ

- All three are **the same formula** with different weights, and all three are **plug-in rules**: they estimate η and imitate $f^\star$, minimising no error over $\mathcal{H}$
- All three have **exactly one** knob controlling complexity, and it must be chosen by **validation or cross-validation**, exactly as in Lecture 3. Choosing it by E_{in} would always pick the least smoothing

Decision trees and random forests

A tree and the partition it produces





The XOR data, with a tree of **three** splitting nodes and **four** leaves

Each leaf is one rectangle of the picture on the left

- Each leaf is a **logical AND** of the decisions on the path to it, and each class is the **OR** of its leaves:

$$\text{class 1}: \quad \big([x_1 \leq 0] \text{ AND } [x_2 \leq 0]\big) \text{ OR } \big([x_1 > 0] \text{ AND } [x_2 > 0]\big)$$

- **That sentence is the classifier.** Nothing was hidden in model parameters or training data points location

Source: Braga-Neto (2024), Figure 6.10 (Example 6.6).

Binary questions

- **The idea**: classify by asking a sequence of simple questions, each one narrowing the region of feature space we are in: "is $x_3 \leq 0.4$?", then "is $x_1 \leq 2$?", and so on
- A **tree classifier** is a hierarchy of **splitting nodes**, where a question is asked, ending in **leaf nodes**, where a label is assigned. The first node is the **root**; the **depth** is the largest number of questions on any path
- The leaves **partition the feature space**: they do not overlap and they cover everything, so we are back to a partition, as in the histogram rule, but now the partition is *chosen using the data*
- Two things trees give:
  - they handle **categorical** features as naturally as numerical ones
  - they are **interpretable**: the classifier can be read out as logical rules and checked by a domain expert
- Unlike Chapters 4 and 5, **no distribution is estimated**: the partition is chosen directly, by making an empirical criterion small. This is the third family of the lecture

The hypothesis set of a decision tree

- We consider the **CART** (**C**lassification **A**nd **R**egression **T**ree) algorithm, where every question has the form "is $x_j \leq \alpha$?" for one coordinate j and one threshold α
- Then every region is a **rectangle** with sides parallel to the axes, and

$$\mathcal{H} \;=\; \big\{\text{classifiers constant on each cell of an axis-aligned rectangular partition}\big\}$$

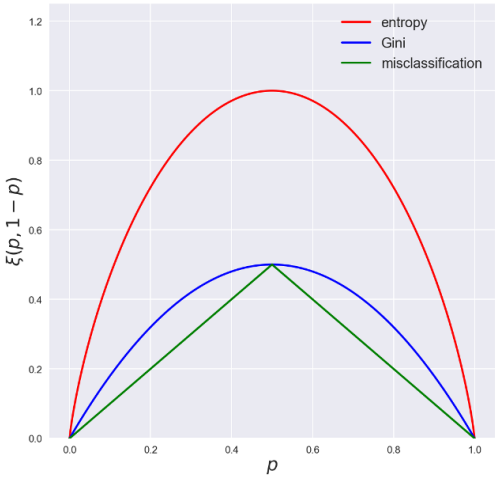
- If we allow trees of unlimited depth, $\mathcal{H}$ is **enormous**: with enough splits every training point can be isolated in its own leaf, so $E_{\text{in}} = 0$ always. In the language of Lecture 3, such a model is **unfalsifiable**
- Limiting the depth, or the number of leaves, or the number of points per leaf, cuts $\mathcal{H}$ down to something learnable. **Trees must be constrained, and the constraints is the model-selection parameter**
- Note the family resemblance to the histogram rule, the difference is that here the partition is **built from the data**, so the boundary need not sit on a fixed grid

A criterion, not a loss: impurity

- How is the tree built? By choosing splits that make the resulting regions as **pure** as possible, that is, as close as possible to containing a single class
- For a region R containing $N_0(R)$ and $N_1(R)$ points of each class, let $p = N_1(R)/(N_1(R) + N_0(R))$ be the proportion of points in class one. The **impurity** of R is $I(R) = \xi(p, 1-p)$, where ξ must satisfy
  - $\xi(0,1) = \xi(1,0) = 0$: a region with one class only is **pure**
  - ξ is largest at $p = \frac{1}{2}$: an evenly split region is **maximally impure**
  - ξ increases on $[0, \frac{1}{2}]$ and decreases on $[\frac{1}{2}, 1]$
- Three standard choices:
  - **entropy**: $\xi(p, 1-p) = -p \log p - (1-p) \log(1-p)$
  - **Gini**: $\xi(p, 1-p) = 2p(1-p)$
  - **misclassification**: $\xi(p, 1-p) = \min(p, 1-p)$

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Section 6.3, equation (6.89).

The three impurity functions



- All three vanish at $p = 0$ and $p = 1$ and peak at $p = \frac{1}{2}$; they differ only in *how sharply*
- These are old friends. The **misclassification** impurity $\min(p, 1-p)$ is exactly the error of the best possible classifier from Lecture 2, computed inside the region
- The **entropy** is the information-theoretic uncertainty of the label in the region
- Entropy and Gini are preferred in practice, because they keep highly rewarding splits that the misclassification do not

Source: Braga-Neto (2024), Figure 6.11.

How the tree is grown

- Splitting a region R at coordinate j and threshold α produces $R^j_{\alpha,-}$ and $R^j_{\alpha,+}$. The **impurity drop** is

$$\Delta_R(j,\alpha) \;=\; I(R) \;-\; \frac{N(R^j_{\alpha,-})}{N(R)} I\big(R^j_{\alpha,-}\big) \;-\; \frac{N(R^j_{\alpha,+})}{N(R)} I\big(R^j_{\alpha,+}\big)$$

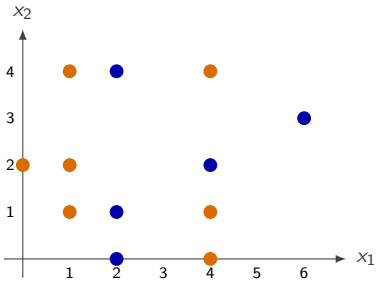
  the impurity we had, minus the average impurity we are left with
- At each node, take the (j,α) with the **largest** impurity drop. There are finitely many distinct splits, so this search can be exhaustive
- Repeat on each child. A node becomes a leaf when it is pure, or when a stopping rule fires

This is *not* ERM

Impurity is not a loss on observations: it is a score for a *region*. And the search is **greedy**: each split is chosen as if it were the last, and is never revisited. The tree returned is not the best tree in $\mathcal{H}$ by any measure. It is the tree this procedure happens to build.

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Section 6.3, equations (6.90)–(6.91).

Growing a tree by hand



- $N = 12$ observations, $d = 2$, both features taking whole-number values
- • is class 1 (5 points), • is class 0 (7 points)
- We use the **Gini** impurity $I(R) = 2p(1-p)$, and the stopping rule $s = 2$: no split may leave fewer than 2 points in a child
- Candidate thresholds are the **midpoints between consecutive observed values** of a feature. For x_1 the observed values are $0, 1, 2, 4, 6$, giving $\alpha \in \{0.5,\ 1.5,\ 3,\ 5\}$

These points are nowhere near linearly separable. Let us see what CART does with them.

Step 1: the root

- At the root, $N_1 = 5$ and $N_0 = 7$, so $p = \frac{5}{12}$ and

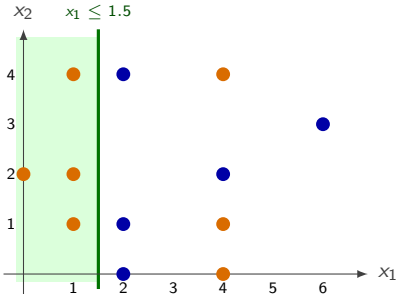
$$I(\text{root}) \;=\; 2 \cdot \tfrac{5}{12} \cdot \tfrac{7}{12} \;=\; \tfrac{35}{72} \;\approx\; 0.486$$

- Now try every candidate split and compute Δ. Splits leaving fewer than $s = 2$ points on a side are not allowed:

split	$\lvert R_- \rvert$	$\lvert R_+ \rvert$	$I(R_-)$	$I(R_+)$	Δ
$x_1 \le 0.5$	1	11		*not allowed*: $1 < s$	
$x_1 \le 1.5$	4	8	0	$15/32$	$\mathbf{25/144 \approx 0.174}$
$x_1 \le 3$	7	5	0.490	0.480	0.000
$x_1 \le 5$	11	1		*not allowed*: $1 < s$	
$x_2 \le 0.5$	2	10	0.500	0.480	0.003
$x_2 \le 1.5$	5	7	0.480	0.490	0.000
$x_2 \le 2.5$	8	4	0.469	0.500	0.007
$x_2 \le 3.5$	9	3	0.494	0.444	0.005

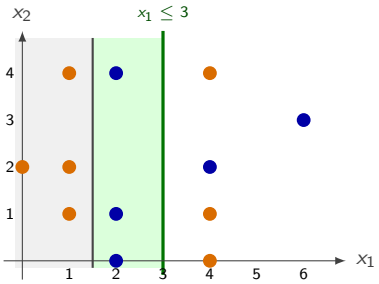
- The winner is $x_1 \le 1.5$, and by a wide margin. It isolates four points that are *all* class 0, so $I(R_-) = 0$

Step 1, in the picture

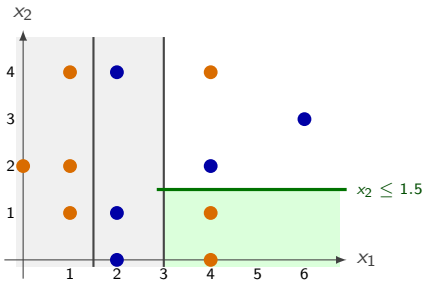


- **Left of the line**: 4 points, and they are **all orange**. A pure region, $I = 0$
- **Right of the line**: the remaining 8, still mixed: 5 blue and 3 orange
- That is why this split wins. No other cut of this data isolates a pure group of four
- The left child is pure, so it becomes a **leaf labelled** 0 immediately. The right child is passed back to the same procedure

Steps 2 and 3, in the picture



Step 2, on the 8 points to the right of the grey line.



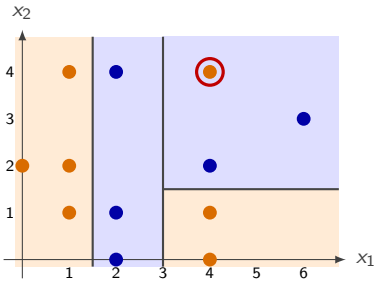
Step 3, on the 5 points remaining on the right.

- **Step 2** peels off the three points with $x_1 = 2$, all blue: another pure leaf. $\Delta = \frac{15}{32} - \frac{5}{8} \cdot \frac{12}{25} = \frac{27}{160} \approx 0.169$
- **Step 3** peels off $(4, 0)$ and $(4, 1)$, both orange: a third pure leaf. $\Delta = \frac{12}{25} - \frac{3}{5} \cdot \frac{4}{9} = \frac{16}{75} \approx 0.213$
- What is left is $(4, 2)$ and $(6, 3)$ in blue and $(4, 4)$ in orange. **Three points, and mixed**

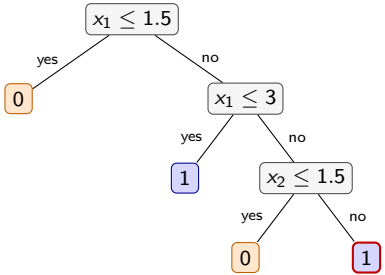
Where the algorithm stops

- That last node has 3 points and is *not* pure, so we would like to split it again. **But we cannot**: any split of three points leaves 1 or 2 on one side, and $1 < s = 2$
- So it becomes a leaf, labelled by its **majority class**, which is 1 (two against one). The point $(4, 4)$ is therefore **misclassified by the very tree it helped to build**
- The tree has 4 leaves and
$$E_{\text{in}}(\hat{h}) \;=\; \frac{1}{12} \;\approx\; 0.083$$
- **This is the point of s.** With $s = 1$ the algorithm would have carried on, isolated $(4, 4)$ in a leaf of its own, and reported $E_{\text{in}} = 0$, a fully-grown tree memorising a single point
- Which of the two is the better *classifier* is not decided by E_{in}, and cannot be: it is a model-selection question, settled by validation

The finished tree and its partition



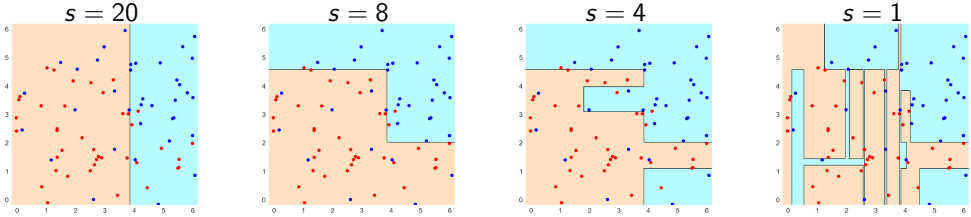
Shaded by predicted class. The point ringed in **red** is the single training error.



Three questions, four leaves. The red leaf is the impure one.

- Read the classifier out loud: *"x_1 small: class* 0. *x_1 middling: class* 1. *x_1 large: class* 0 *if x_2 is small, class* 1 *otherwise."*

Trees overfit



- CART with the Gini impurity; no split may leave fewer than s points in a child
- $s = 1$ is the **fully-grown** tree: every leaf pure, $E_{\text{in}} = 0$, and a mess of slivers around individual points
- $s = 20$ leaves a single split. Such one-split trees are called **stumps**, and are often surprisingly effective
- **Fully-grown trees should never be used.** Besides s one can limit the depth or the number of leaves, or **prune**: grow fully, then merge back leaves

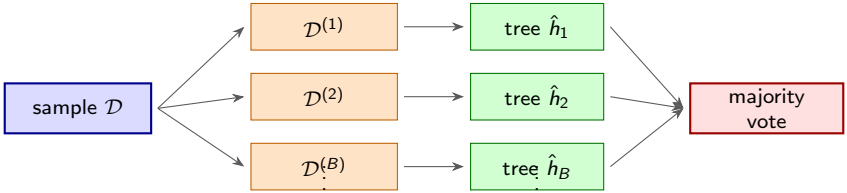
Source: Braga-Neto (2024), Figure 6.12 (Example 6.7).

What the tree hypothesis means

- $\hat{h}$ is a **partition of the feature space into axis-aligned boxes, each labelled by the majority class of the training points inside it**, with the boxes chosen greedily to separate the classes as fast as possible
- It is a histogram rule whose grid was *based on data* rather than fixed in advance
- Its strengths: interpretability, categorical variables, no scaling of the features needed (each split looks at one variable at a time), and automatic disregard of irrelevant variables
- Its weaknesses: boundaries are staircases, so a diagonal boundary is described badly; and the tree is **very unstable**: change one observation and a split near the root may flip, rewriting the entire tree below it
- That instability is a symptom of **high variance**

Bagging: averaging away the variance

- A rule with low bias and high variance is roughly right on average but wildly sensitive to the sample. Averaging many such rules should help, but we have only **one** sample
- **The bootstrap**: make B new samples by drawing N observations from $\mathcal{D}$ **with replacement**. Each has the size of $\mathcal{D}$, but some observations appear twice and others not at all



- Fit the rule on each bootstrap sample and let the B classifiers **vote**: this is **bagging** (bootstrap aggregating)
- The bias is unchanged; the variance is **substantially reduced**, because the idiosyncrasies of the individual fits disagree and are outvoted

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Sections 3.5.1 and 6.3.

Random forests

Random forest

Grow a large number of **fully-grown** trees, each on a bootstrap sample of $\mathcal{D}$, and each restricted to consider only a **random subset of the variables at every split**. Classify by majority vote.

- Two sources of randomness: the bootstrap decorrelates the trees through the *data*; the random subset of variables decorrelates them through the *splits*
- Without the second device a single dominant variable would sit at the root of nearly every tree, and the trees would be near-copies and averaging near-copies achieves nothing
- The trees are grown **fully**, on purpose: each should have low bias, and the vote removes the variance. *Individually they overfit; collectively they do not*
- $\hat{h}$ means: **the majority opinion of many different, individually overfitted trees**
- The price is the one thing a single tree was best at: **a forest is not interpretable**

A forest on the twelve points

- Back to our 12 observations. Draw 12 of them **with replacement**, three separate times. Here is what actually comes out:

	what was drawn (a superscript is a repeat)	**left out**	**sample variable**	**root split**
$\mathcal{D}^{(1)}$	$(0,2)\ (1,1)\ (1,2)\ (1,4)\ (2,0)\ (2,1)\ (2,4)^4\ (4,1)\ (4,2)$	3 points	x_1	$x_1 \le 1.5$
$\mathcal{D}^{(2)}$	$(1,4)^2\ (2,0)^3\ (2,1)^3\ (4,0)\ (4,2)\ (4,4)\ (6,3)$	5 points	x_2	$x_2 \le 3.5$
$\mathcal{D}^{(3)}$	$(0,2)\ (1,1)\ (1,2)^3\ (1,4)\ (2,1)\ (2,4)\ (4,2)^2\ (4,4)\ (6,3)$	3 points	x_1	$x_1 \le 1.5$

- Each resample has 12 points but only 7 to 9 *distinct* ones. In $\mathcal{D}^{(1)}$ the point $(2,4)$ was drawn **four times**, and three observations never appeared at all

- **The trees genuinely differ**: two of them open with $x_1 \le 1.5$ when x_1 is the sample variable, but $\mathcal{D}^{(2)}$ opens on the *other* feature that was the one sample there

And what the vote does

- To classify a point, ask all B trees and take the **majority**. Take $(4, 4)$, the point the single tree got wrong: **all three** of these trees put it in class 0, which is correct
- That is bagging working exactly as advertised: the single tree's mistake came from a leaf built on 3 points, and resampling breaks up that particular accident

What averaging buys, and what it does not

With 12 points, no method will be impressive, and E_{in} **was never the thing to watch**. What the forest buys is **stability**: move one observation and the single tree can rewrite itself completely, while the vote of hundreds of trees barely twitches. That is variance reduction, and it is paid for with interpretability: the forest cannot be read out as rules.

Regression

The regression framework

- Now Y is a **number**, and the loss is the squared loss $\ell((x, y), h) = (h(x) - y)^2$
- From Lecture 2, the best possible predictor and its error are

$$f^\star(\mathbf{x}) = \mathbb{E}[Y \mid \mathbf{X} = \mathbf{x}], \qquad E_{\text{out}}(f^\star) = \mathbb{E}\big[\text{Var}(Y \mid \mathbf{X})\big]$$

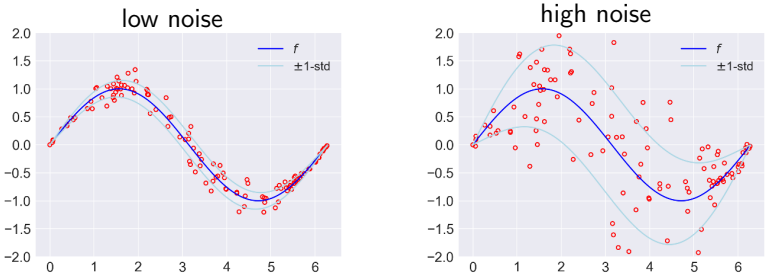
- It is convenient to write the phenomenon as "signal plus noise":

$$Y \;=\; f^\star(\mathbf{X}) + \varepsilon, \qquad \mathbb{E}[\varepsilon \mid \mathbf{X}] = 0$$

which is not an assumption: it is just the definition of $\varepsilon = Y - f^\star(\mathbf{X})$

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Section 11.1, Theorem 11.2.

How much noise there is, changes everything



- Same $f^\star(x) = \sin(x)$, same inputs, two noise levels. The solid curve is $f^\star$, the band one standard deviation of the noise
- **Left**: knowing x tells us almost exactly what y will be
- **Right**: knowing x still helps, but much of y remains unpredictable. $E_{\text{out}}(f^\star)$ is large, and **no method can do better**. Any assessment must be read against this floor

Source: Braga-Neto (2024), Figure 11.1 (Examples 11.1–11.2).

Parametric regression, and least squares

- Assume a **shape** with finitely many unknowns:

$$Y \;=\; h(\mathbf{X};\boldsymbol{\theta}) + \varepsilon, \qquad \boldsymbol{\theta} \in \Theta \subseteq \mathbb{R}^m, \qquad \mathcal{H} = \{h(\cdot\,;\boldsymbol{\theta}) : \boldsymbol{\theta} \in \Theta\}$$

- Estimate $\boldsymbol{\theta}$ and return the plug-in predictor $\hat{h}(\mathbf{x}) = h(\mathbf{x};\hat{\boldsymbol{\theta}})$. But **how** is $\boldsymbol{\theta}$ estimated?
- The gap between what the predictor says at $\mathbf{X}_i$ and what we observed is the **residual** $Y_i - h(\mathbf{X}_i;\boldsymbol{\theta})$
- **Least squares** makes the squared residuals as small as possible in total:

$$\hat{\boldsymbol{\theta}} \;=\; \arg\min_{\boldsymbol{\theta}\in\Theta} \underbrace{\sum_{i=1}^{N} \big(Y_i - h(\mathbf{X}_i;\boldsymbol{\theta})\big)^2}_{\text{residual sum of squares, RSS}}$$

Least squares *is* ERM

Divide the RSS by N and it is exactly $E_{\text{in}}(h)$ under the **squared loss** $\ell((\mathbf{x},y),h) = (h(\mathbf{x})-y)^2$, so least squares *is* ERM

Linear regression, and what "linear" means

- The basic model is a hyperplane in the features,

$$Y = \theta_0 + \theta_1 X_1 + \cdots + \theta_d X_d + \varepsilon$$

- but the name is misleading, because the model extends immediately to

$$Y = \theta_0 \phi_0(\mathbf{X}) + \theta_1 \phi_1(\mathbf{X}) + \cdots + \theta_k \phi_k(\mathbf{X}) + \varepsilon$$

where the **basis functions** ϕ_j can be anything at all: powers, logarithms, sines, indicator functions, products of variables

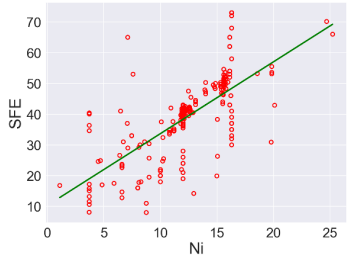
Linear in the *parameters*, not in the variables

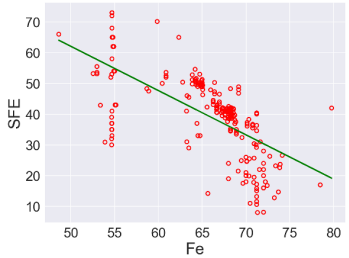
What must be linear is the dependence on $\boldsymbol{\theta}$. The dependence on $\mathbf{x}$ can be as curved as we like: **transform the variables, then be linear**.

- **Polynomial regression** is the standard example: $\phi_j(X) = X^j$, giving $Y = \theta_0 + \theta_1 X + \cdots + \theta_k X^k + \varepsilon$, with the order k playing the role of the complexity knob

Reference: Braga-Neto (2024), *Fundamentals of Pattern Recognition and Machine Learning*, Section 11.3.1, equations (11.19)–(11.21).

Linear regression on real data





- Stacking fault energy of a steel alloy, regressed separately on its Iron and its Nickel content
- The fitted slopes have signs that a metallurgist can check: energy **decreases** with more Iron, **increases** with more Nickel
- Note the scatter. The fit describes **average** behaviour, not a prediction to trust for one alloy

Source: Braga-Neto (2024), Figure 11.3 (Example 11.4).

Nonparametric regression: local averaging

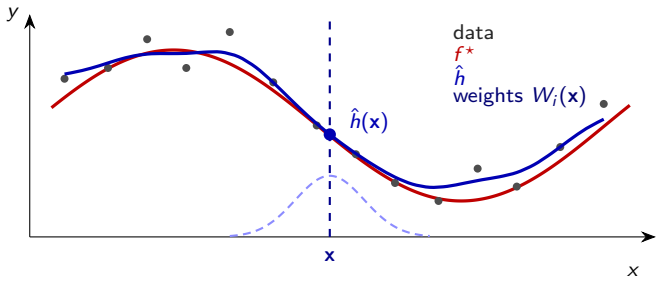
- Drop the parametric family and estimate $f^\star(\mathbf{x}) = \mathbb{E}[Y \mid \mathbf{X} = \mathbf{x}]$ by **averaging the Y_i of the observations near x**

Nadaraya–Watson kernel regression

$$\hat{h}(\mathbf{x}) \;=\; \frac{\sum_{i=1}^{N} \kappa\!\left(\dfrac{\mathbf{x} - \mathbf{X}_i}{\sigma}\right) Y_i}{\sum_{i=1}^{N} \kappa\!\left(\dfrac{\mathbf{x} - \mathbf{X}_i}{\sigma}\right)} \;=\; \sum_{i=1}^{N} W_i(\mathbf{x})\, Y_i$$

- **The weights $W_i(\mathbf{x})$ are exactly those of kernel classification**; only what is averaged changed: the labels $\mathbb{1}\{Y_i = 1\}$ there, the numbers Y_i here
- Same kernels, same bandwidth: small σ overfits, large σ flattens everything towards the average of Y
- Using "the k nearest neighbours" instead gives **kNN regression**: the average Y of the k closest observations

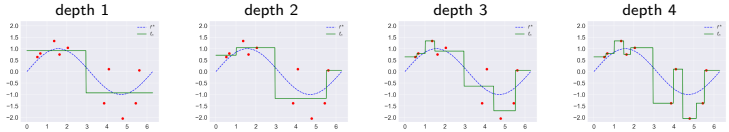
What local averaging looks like



- To predict at **x**: put a bump of width σ over **x**, weight every observation by its height under the bump, and take the **weighted average of their y-values**
- Do this at every **x** and the predictions trace out a curve
- $\hat{h}$ means: **a local weighted average of the observed outcomes**. There is no formula, no coefficient to interpret: again, the data *is* the model
- Where data is sparse, the average is taken over very few points and the curve becomes unreliable

Regression trees, and forests

- To turn a classification rule into a regression rule: **average outcomes instead of majority vote** and **replace the criterion by the RSS**
- For **CART** that is a small change to what we did by hand: a leaf is labelled by the **average response** of the points inside it rather than their majority class, and a split minimises the **RSS of the two children** rather than their impurity. The greedy search, the axis-aligned questions and the rule s are unchanged



- The fit is **piecewise constant** (one level per leaf)
- Too little depth underfits, too much chases the noise. **Random forests** apply unchanged, with the B trees **averaged** rather than voted

Source: Braga-Neto (2024), Figure 11.10 (Example 11.11).

We need linear algebra and calculus

Where we are

- Lectures 2 and 3 settled the question of **whether** learning is possible. With enough data relative to the complexity of $\mathcal{H}$, E_{in} is close to E_{out}, and minimising the first is a sound way to pursue the second
- This lecture supplied some **techniques**: three families of rules, and for each, a hypothesis set, a criterion, and a knob to be set by validation
- So we know that learning is feasible, and we know some ways to attempt it. **What is missing is how to do it in practice**
- Almost every method here ended at the same place: *"and now minimise this"* with the cross-entropy loss, the impurity of a split, the residual sum of squares, etc

Next steps

Actually **computing minimisers**. That is optimisation, and to study it we first need the two languages it is written in: **linear algebra** and **calculus**. They are the subject of the next parts of the course, so we can tackle optimisation in weeks 11 and 12

Lecture highlights

- **Three families**. **Parametric** rules assume a shape and estimate a few numbers; **nonparametric** rules assume only locality and smooth; **tree-based** classification
- **Plug-in rules minimise nothing**. Chapters 4 and 5 estimate η and imitate $f^\star$
- **New losses**: cross-entropy for probability models, impurity for regions; **surrogates** for the binary loss we actually care about
- **The same trade-off everywhere**: every picture this week was bias against variance

Next: *the linear algebra, and then calculus (week 10.5), we need to actually carry out the minimisation.*

References

- Braga-Neto, U. M. (2024). *Fundamentals of Pattern Recognition and Machine Learning*. 2nd edition. Springer. **Chapters 4, 5, 6 and 11**.
- Abu-Mostafa, Y., Magdon-Ismail, M., & Lin, H.-T. (2012). *Learning from Data*. AMLBook.
- Devroye, L., Györfi, L., & Lugosi, G. (1996). *A Probabilistic Theory of Pattern Recognition*. Springer.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning*. 2nd edition. Springer.